

没有 datasheet, 怎么种 Linux 给 (传说中的) AIPC 移植 Linux

喵喵 catme0w



AIPC 的电视广告



传诵至今的鬼畜素材

但几乎没有人见过真机



广告里这些怪异宣传，谜底更难考据

同年，电视上还有另一种广告……

「买无线网卡送笔记本电脑」

你信吗？700多元买无线网卡送笔记本电脑

<http://www.sina.com.cn> 2010年04月14日05:09 重庆时报

本报讯（记者 邓姣）“一款比500万像素相机还清晰的高清相机，仅售376元！”“24K黄金镶钻手机，只需998元！”……看到这样的广告，谁不心动？如果是，请你先耐心看看市消委会的这份“电视购物”消费现状分析。

“花700多元，就能买回一张无线网卡，还免费送一个笔记本电脑！”前段时间，刘小姐就被电视屏幕上的花言巧语所蛊惑，当她收到货后才发现自己上当了。买回来的电脑根本不能无线上网，用网线上网也经常掉线，更夸张的是，上网时只能打开一个网页，多了就会死机，连QQ也不能用。

几乎无人知晓，我只找到这一篇打假报道



?

你信吗？700多元买无线网卡送笔记本电脑

<http://www.sina.com.cn> 2010年04月14日05:09 重庆时报

(记者 邓姣) “一款比500万像素相机还清晰的高清相机，仅售376元！只需998元！”……看到这样的广告，谁不心动？如果是，请你先耐心看“电视购物”消费现状分析。

多元，就能买回一张无线网卡，还免费送一个笔记本电脑！”前段时间，上的花言巧语所蛊惑，当她收到货后才发现自己上当了。买回来的电脑用网线上网也经常掉线，更夸张的是，上网时只能打开一个网页，多了也不能用。

它们有关系吗？



你信吗？700多元买无线网卡送笔记本电脑

<http://www.sina.com.cn> 2010年04月14日05:09 重庆时报

（记者 邓姣）“一款比500万像素相机还清晰的高清相机，仅售376元！只需998元！”……看到这样的广告，谁不心动？如果是，请你先耐心看“电视购物”消费现状分析。

多元，就能买回一张无线网卡，还免费送一个笔记本电脑！”前段时间，上的花言巧语所蛊惑，当她收到货后才发现自己上当了。买回来的电脑用网线上网也经常掉线，更夸张的是，上网时只能打开一个网页，多了也不能用。

无论你买的是哪个
收到的都是完全相同的一款山寨上网本

**这些信息都失传了这么多年
我怎么能确定？**



我就是当年那个买家



大约七寸的山寨上网本，没有品牌



展台区有实机展出，欢迎体验

Windows CE 5.0



一直以来，都有一个心愿……

**能不能把它从 WinCE
换成 Linux?**

海外同模具的上网本以 Sylvania (等) 品牌销售 已有人成功移植 Linux



难道说?

但主线内核直接支持 Sylvania 的处理器

- Display: HL 070 TN92 – Unknown Manufacturer – 800×480 May be TFT?
- Battery: Unknown Metal, 1800mAh 8.4v
- Charger: 9VDC, 1500mA positive tip.
- System Processor: WonderMedia WM8505+ (linux identifies as an ARM936EJ-S rev 5), unknown speed. Research points anywhere from 200MHz to 600MHz.
- System RAM: 128MB SDRAM, unknown manufacturer. Chip number: NY64X161043
- System Flash: 2GB cumulative (WinCE partitions 500MB for system and 1.5G for storage) – Samsung K9CAG08U10

和 AIPC 完全不同

安凯微电子 AK7802



ARM926EJ-S 核心，一颗为廉价 MP4 设计的芯片

没有芯片手册
没有 datasheet
没有评估板
《啥都没有》

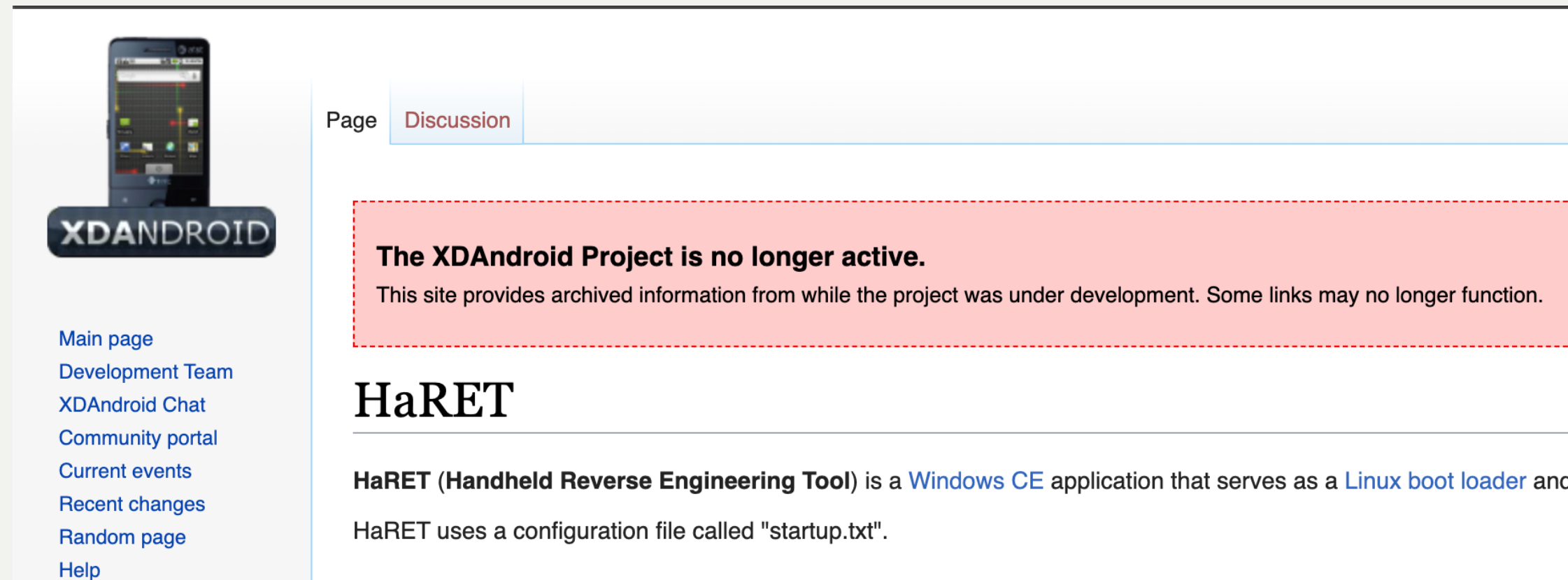
唯一找到的资料是神秘网站上 (疑似) 这台机器的原理图，但无从验证



只有这些东西的话

完全没有办法移植 Linux 啊

2009 年



当年，人们用它给 Windows Mobile 手机装 Android
从 WinCE 用户态直接引导 Linux，不需要刷机



对于一个对硬件一无所知的人来说
这个方案可能是唯一的突破口

但是……

**内核哪里来？
引导参数哪里来？**

遗憾离场……

直到今年

Search all of GitHub

在 GitHub 搜索 AK7802

没有奇迹

```
✓ stevezuo/ak98_kernel · arch/arm/mach-ak88/clock.c  
  
529         clk_cpu.rate =clk_cpu.rate*1000*1000;  
530  
531     #else  
532     #error "Only support AK780X & AK88."  
533     #endif  
534  
535     /* register our clocks */
```

但发现了后继型号 AK98 系列的
Linux 2.6 内核源码

重新编译出 一模一样的 2.6?

那就没劲了!



Linux 7.0

rc3 发布于 2026-03-08



HaRET 还能直接读写物理内存， 提取设备信息

```
1  # See docs/haret.html for a short description of HaRET Scripting Language
2
3  print "Welcome to Hardware (Handheld?) Reverse Engineering Tool!"
4  print "Some basic info about your PDA:"
5  print "MMU L1 descriptor table address is %08x" MMU
6  print "Video RAM address is %08x" VRAM
7  print "Current Process ID is %d" PID
8  print "CPU identification register (p15 r0) is %08x" CP (15, 0)
9
```

毕竟，HaRET 全称叫 Handheld Reverse Engineering Tool

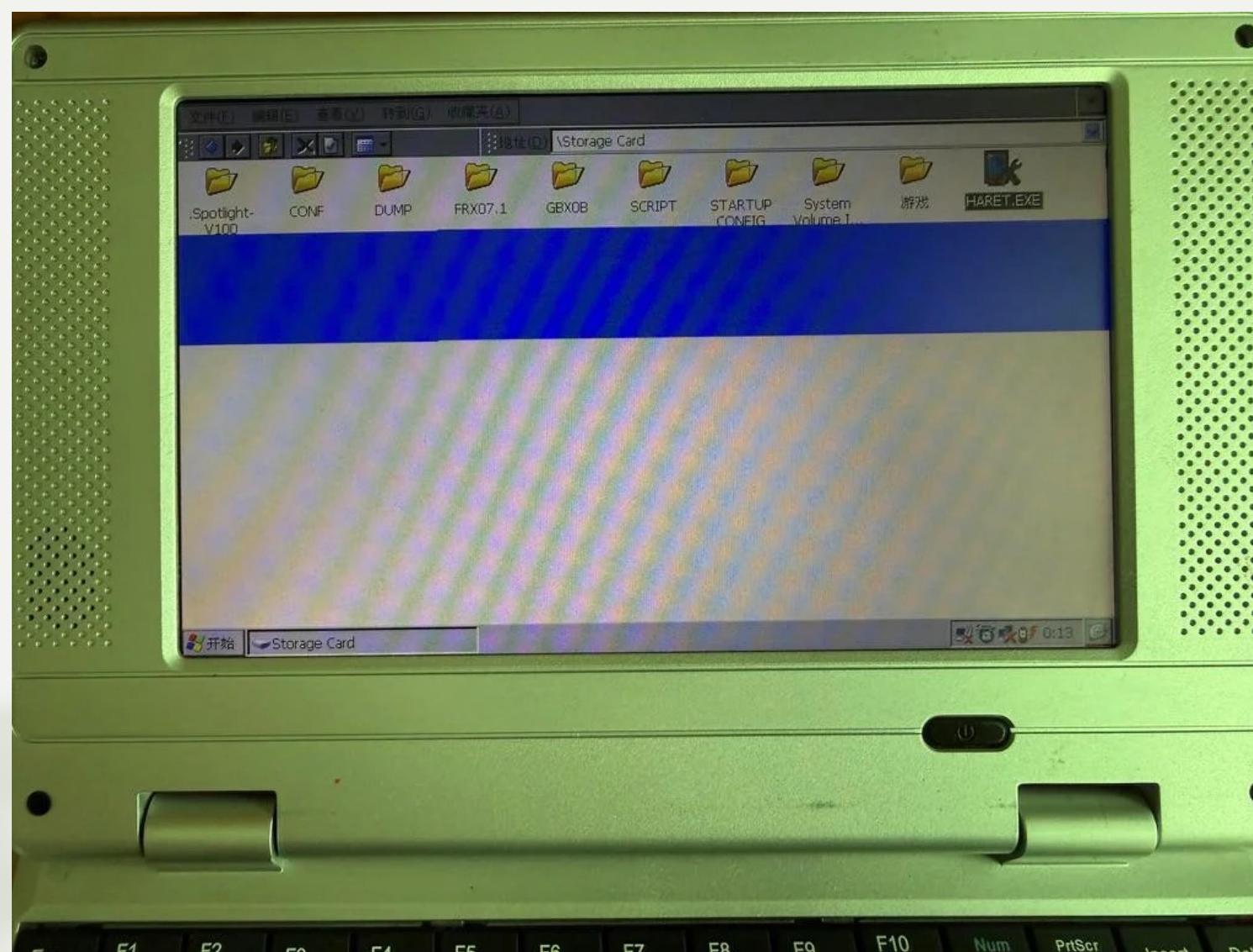
没有 UART，没有 JTAG 屏幕是唯一的调试手段

要调试，得先找 framebuffer

用 HaRET 手动往不同的内存地址写像素 看屏幕会不会出现颜色

```
7  
8   print "Testing 0x33E40000 ..."  
9   PFH 0x33E40000 800*50 0x07E0  
10  
11  print "Testing 0x33E80000 ..."  
12  PFH 0x33E80000 800*50 0x001F  
13
```


找到了!





从 AK98 的 2.6 源码抄出驱动和寄存器表



写成设备树



编译进 7.0



交给 HaRET 引导

但 2.6 和 7.0 天差地别 没那么好抄

屏幕驱动上线之前，如果引导出错，什么都看不到

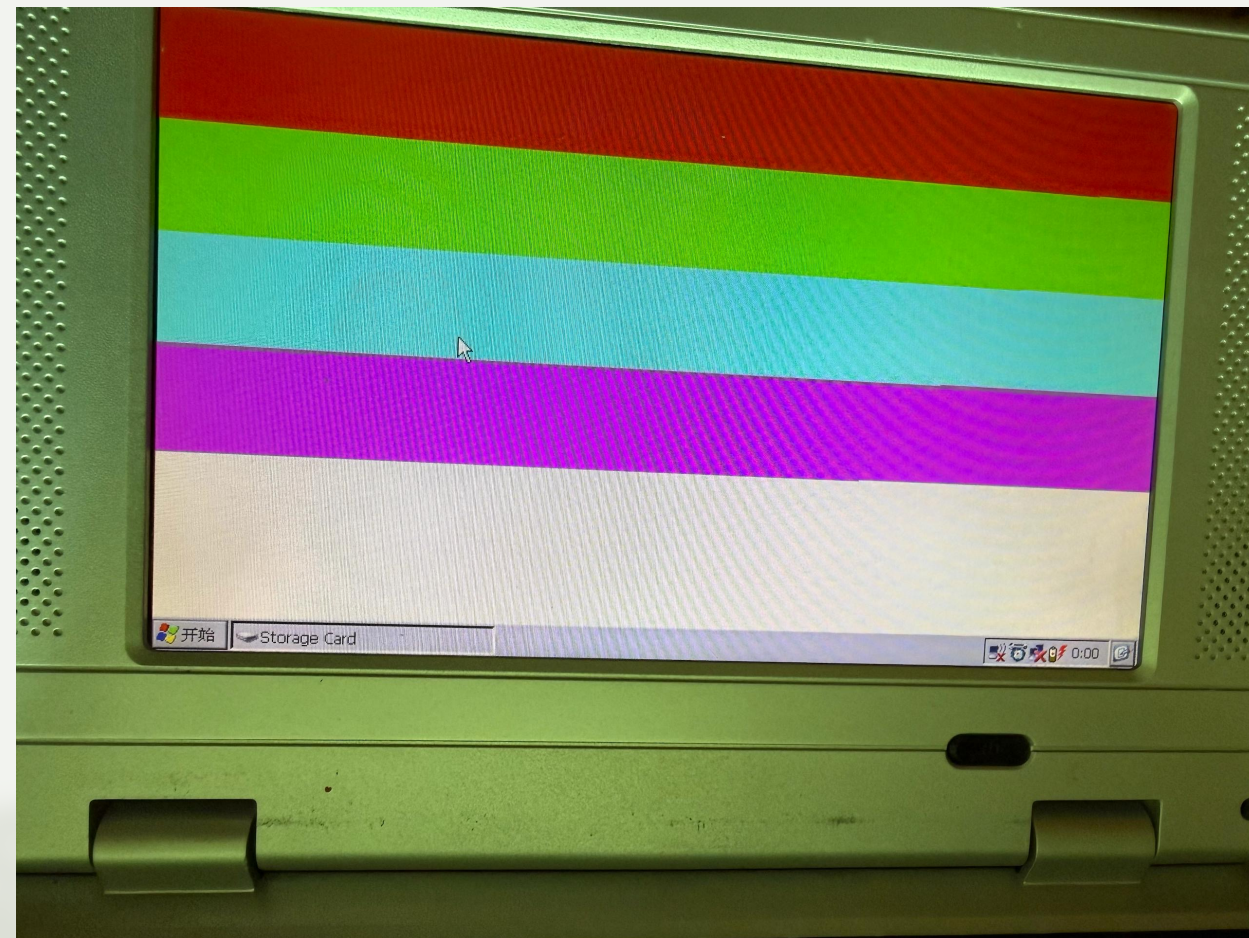


注意到：

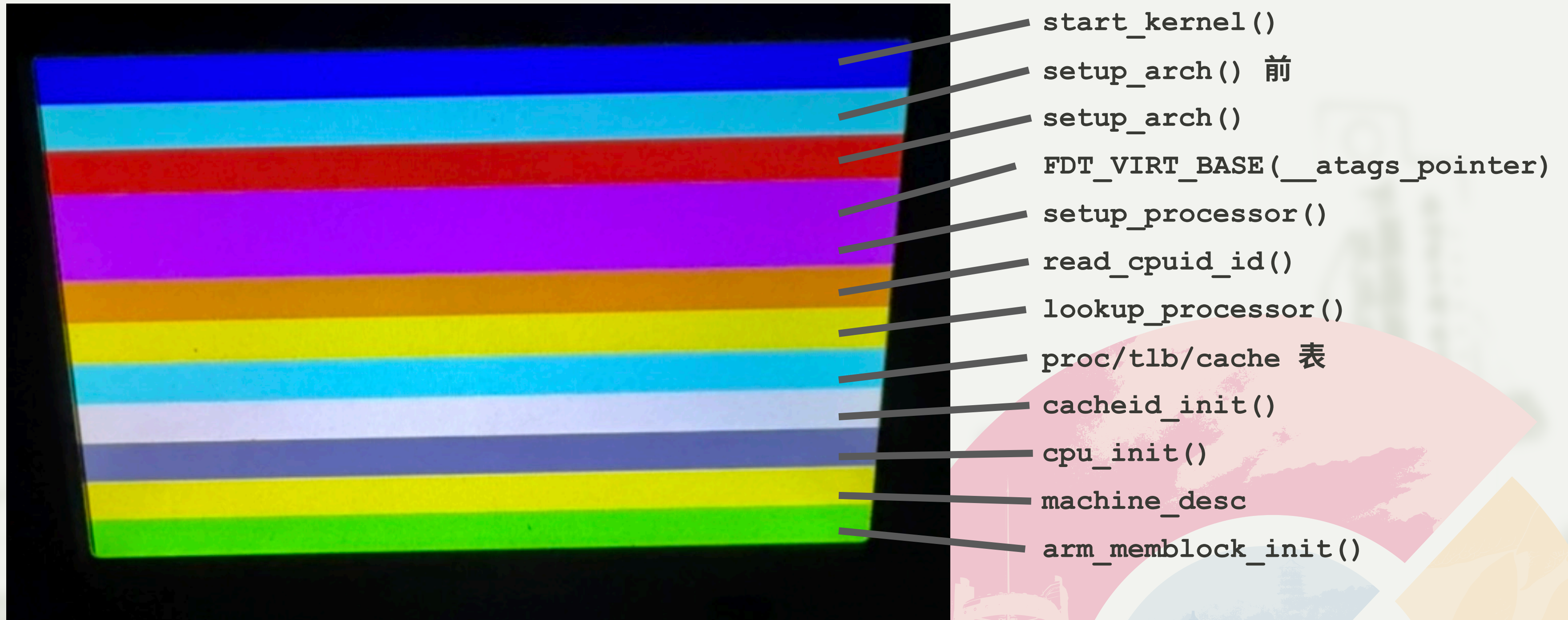
**HaRET 是热启动
LCD 硬件仍保持着 WinCE 初始化好的状态**

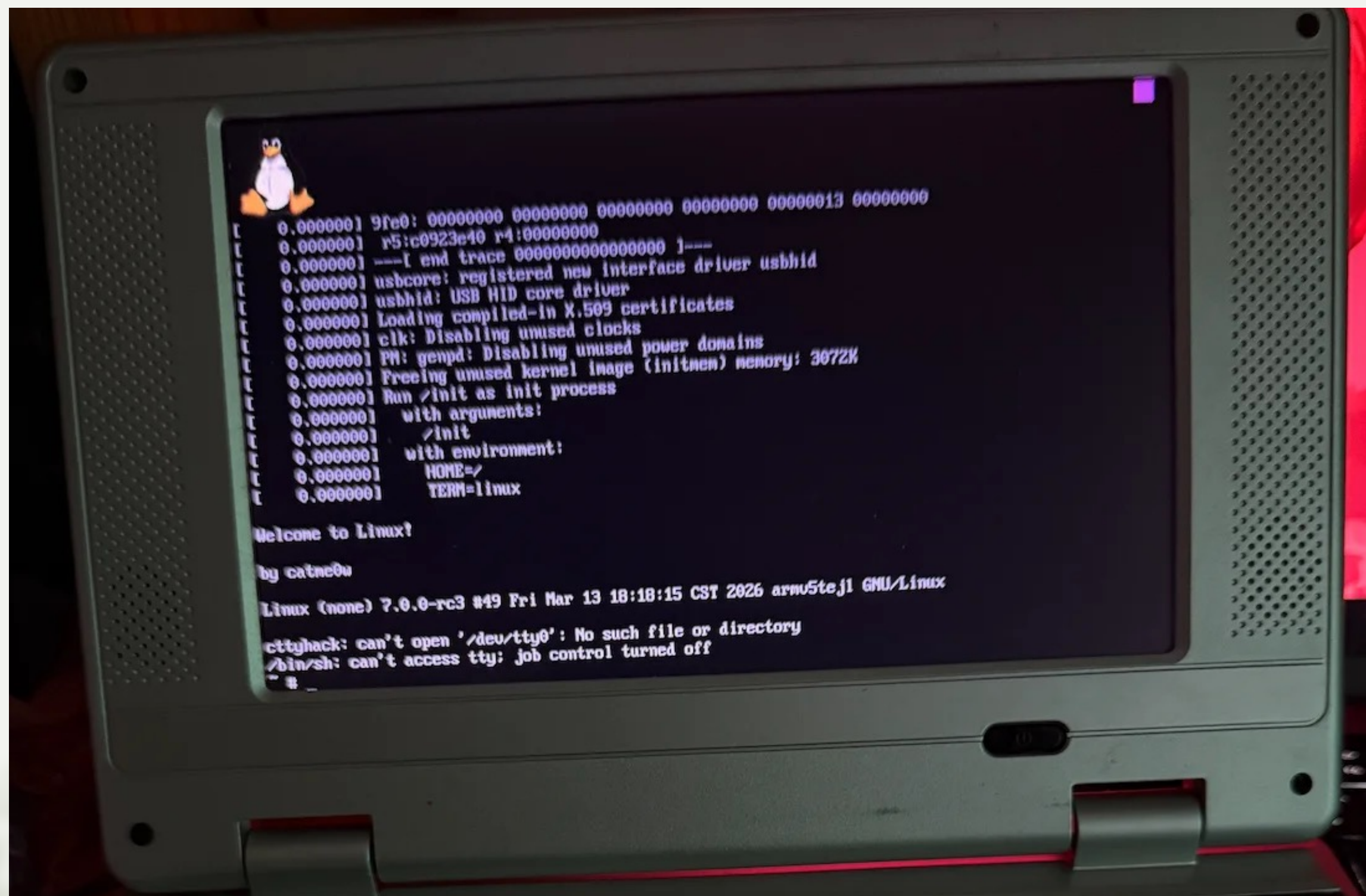
哪怕 CPU 已经移交给 Linux，涂色这一招还能用

给引导流程的每个关键函数插入涂色代码 每个函数涂不同颜色



卡在哪个颜色，就知道卡在哪里

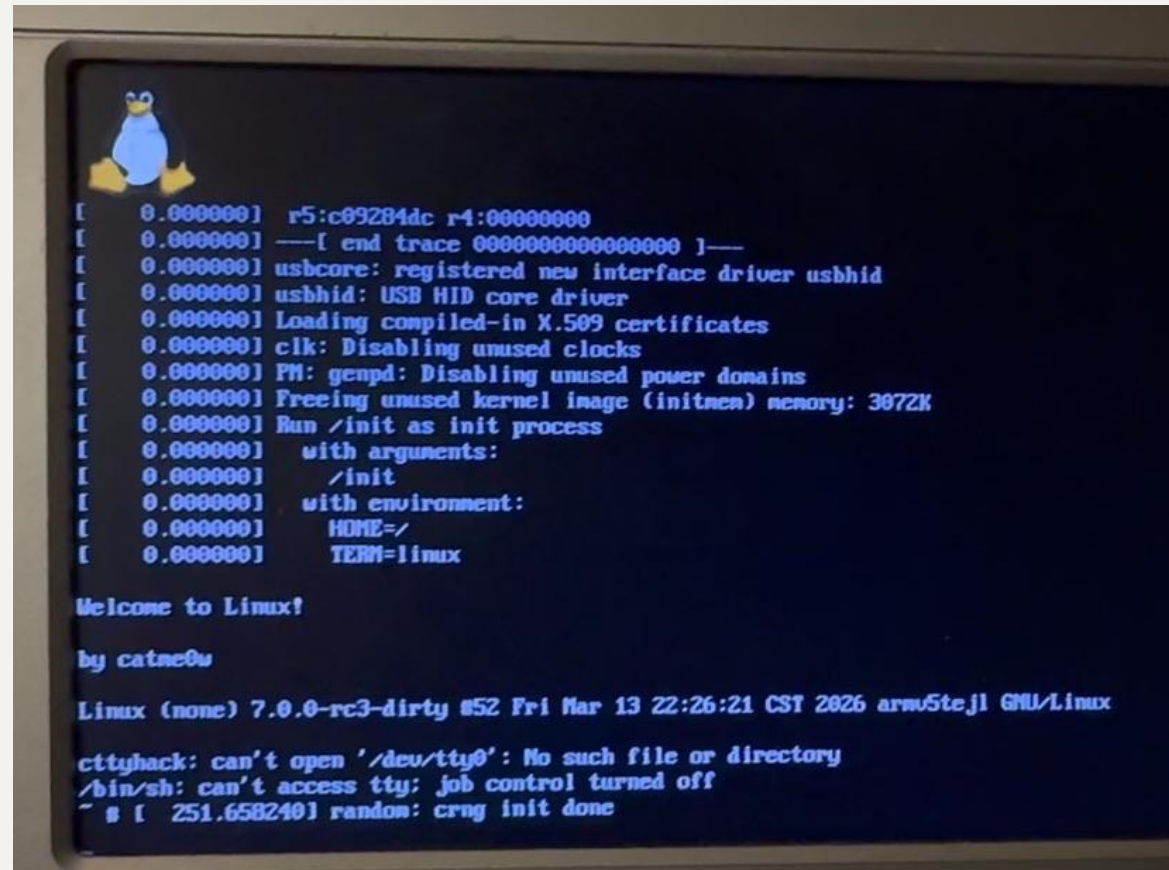




缺少时钟驱动



世界静止了！



```
[ 0.000000] r5:c89284dc r4:00000000
[ 0.000000] —[ end trace 0000000000000000 ]—
[ 0.000000] usbcore: registered new interface driver usbhid
[ 0.000000] usbhid: USB HID core driver
[ 0.000000] Loading compiled-in X.509 certificates
[ 0.000000] clk: Disabling unused clocks
[ 0.000000] PM: genpd: Disabling unused power domains
[ 0.000000] Freeing unused kernel image (initmem) memory: 3872K
[ 0.000000] Run /init as init process
[ 0.000000]   with arguments:
[ 0.000000]   /init
[ 0.000000]   with environment:
[ 0.000000]   HOME=/
[ 0.000000]   TERM=linux

Welcome to Linux!

by catnc0w

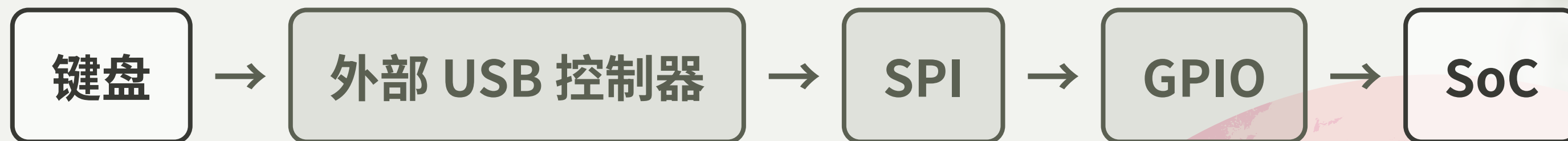
Linux (none) 7.0.0-rc3-dirty #52 Fri Mar 13 22:26:21 CST 2026 armv5tej1 GNU/Linux

cttyhack: can't open '/dev/tty0': No such file or directory
/bin/sh: can't access tty: job control turned off
~ # [ 251.658240] random: crng init done
```

第一版时钟驱动（赶工的），日志大多依旧全是 [0.000000]
但光标闪烁看起来对了

先记作 TODO……

内置键盘是 USB 键盘，但不直连 SoC



💀 必须一次性解决三个子系统

借助 2.6 源码和涂色手法 寄存器表的迷雾已经破开

往后，再不会有什么变数了

往后，再不会有什么变数了

吗？



一个月之后

面对死一般寂静的 SPI 总线

喵喵回想起一个月前

在 AIPC 首次点亮 Linux 7.0 的那个

遥远的下午



↓ 死一般寂静的 SPI 总线

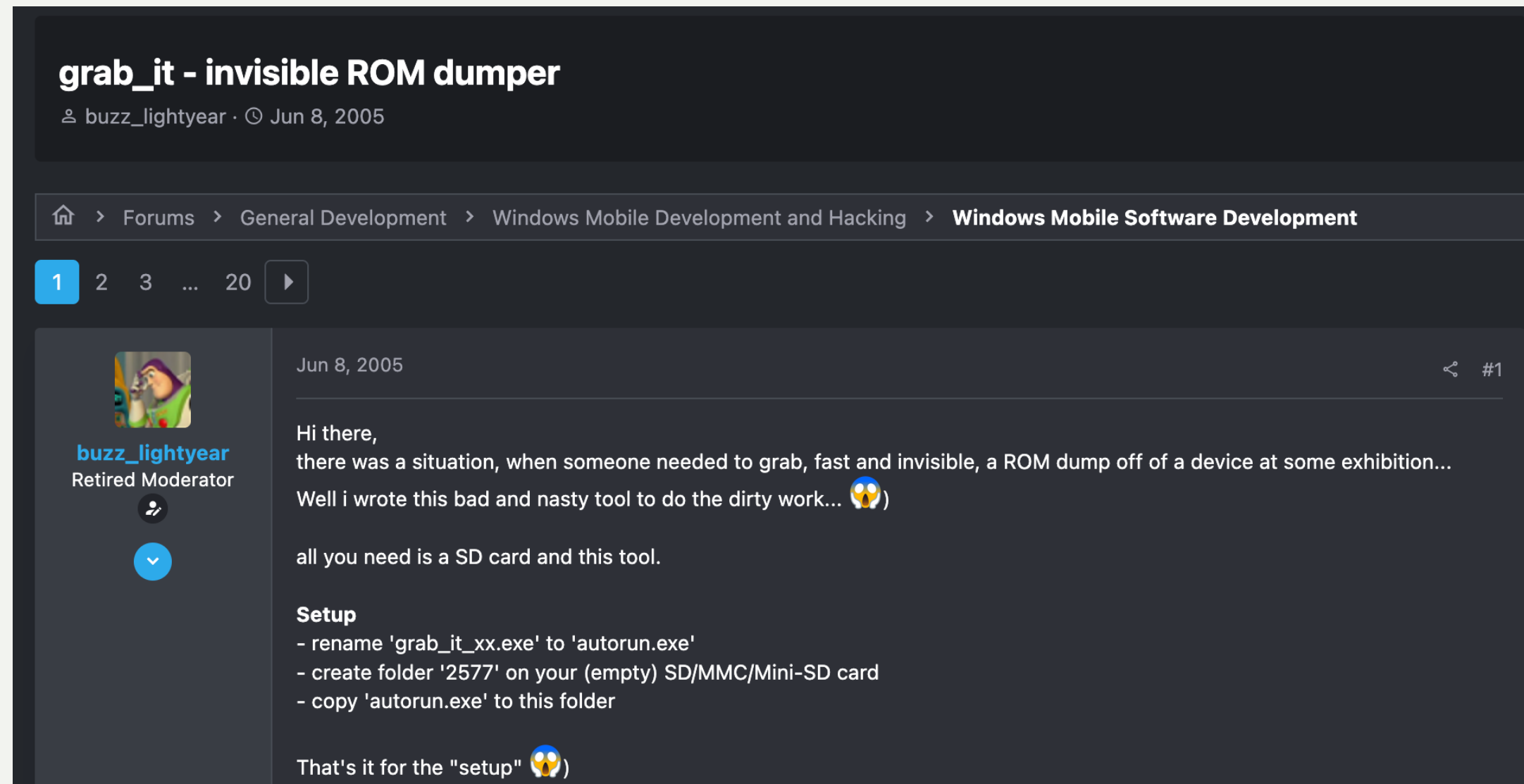
```
[ 0.000000] anyka-gpio 8000074.gpio-controller: registered 117 internal GPIOs
[ 0.000000] ledtrig-cpu: registered to indicate activity on CPUs
[ 0.000000] anyka-bootfb 33ed3c00.framebuffer: cannot reserve framebuffer mem
ory at [mem 0x33ed3c00-0x33f8f3ff]
[ 0.000000] Console: switching to colour frame buffer device 100x30
[ 0.000000] anyka-bootfb 33ed3c00.framebuffer: framebuffer at 0x33ed3c00, 768000 bytes, 800x480x1
6
[ 0.000000] Serial: 8250/16550 driver, 5 ports, IRQ sharing enabled
[ 0.000000] brd: module loaded
[ 0.000000] loop: module loaded
[ 0.000000] ak88-spi 20024000.spi: Anyka SPI controller ready
[ 33.554432] ch374-spi spi0.0: probe scan: mode=0 cs_high=0 reg,cmd=ok:[ff ff ff] cmd,reg=ok:[ff f
f ff]
[ 67.108864] ch374-spi spi0.0: probe scan: mode=3 cs_high=0 reg,cmd=ok:[ff ff ff] cmd,reg=ok:[ff f
f ff]
[ 100.663296] ch374-spi spi0.0: probe scan: mode=1 cs_high=0 reg,cmd=ok:[ff ff ff] cmd,reg=ok:[ff f
f ff]
[ 134.217728] ch374-spi spi0.0: probe scan: mode=2 cs_high=0 reg,cmd=ok:[ff ff ff] cmd,reg=ok:[ff f
f ff]
[ 167.772160] ch374-spi spi0.0: probe scan: mode=0 cs_high=1 reg,cmd=ok:[ff ff ff] cmd,reg=ok:[ff f
f ff]
[ 201.326592] ch374-spi spi0.0: probe scan: mode=3 cs_high=1 reg,cmd=ok:[ff ff ff] cmd,reg=ok:[ff f
f ff]
[ 234.881024] ch374-spi spi0.0: probe scan: mode=1 cs_high=1 reg,cmd=ok:[ff ff ff] cmd,reg=ok:[ff f
f ff]
[ 251.658240] random: crng init done
[ 268.435456] ch374-spi spi0.0: probe scan: mode=2 cs_high=1 reg,cmd=ok:[ff ff ff] cmd,reg=ok:[ff f
f ff]
[ 268.435456] ch374-spi spi0.0: error -ENODEV: unable to find working CH374 SPI profile
```


AK98 总归不是 AK78，2.6 源码终究不能当成可靠信源

**可靠信源只有一个地方：
AIPC 的原厂 WinCE 固件**

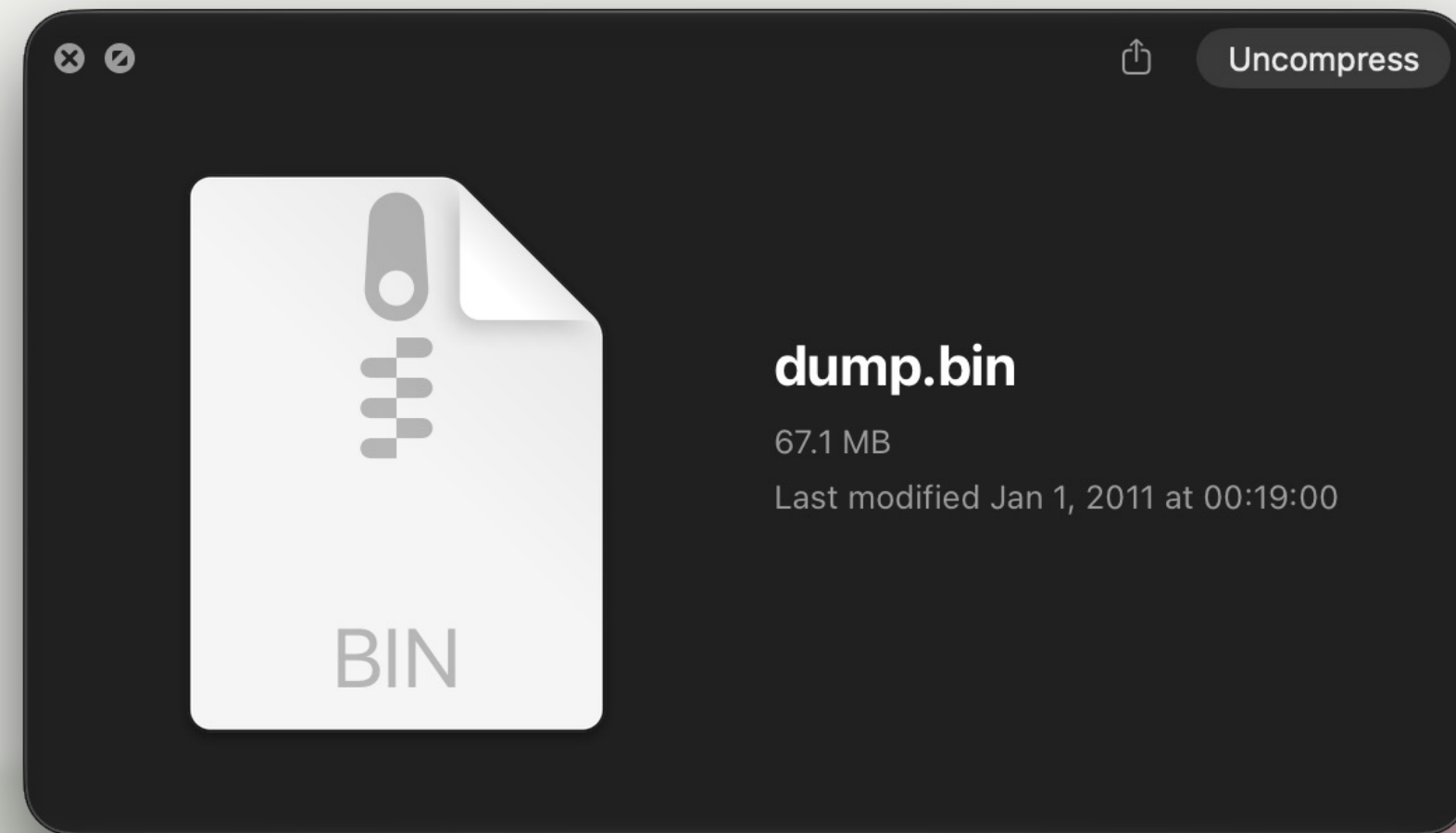
想看 WinCE 的驱动，没那么简单 直接复制系统文件会失败

需要把整个固件 dump 下来

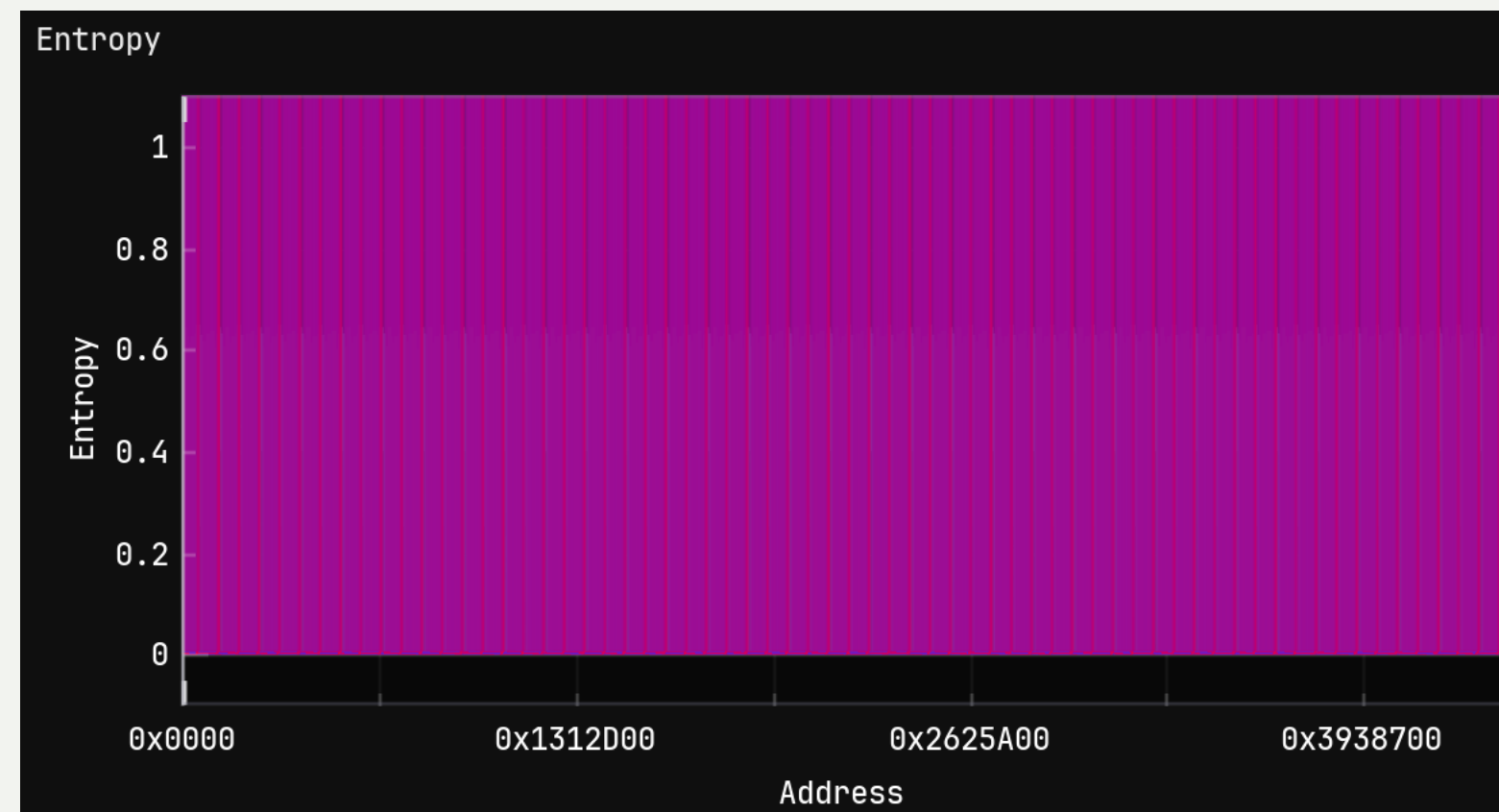


拷进 U 盘，在 WinCE 上执行，
就能自动 dump 固件

奏效了！

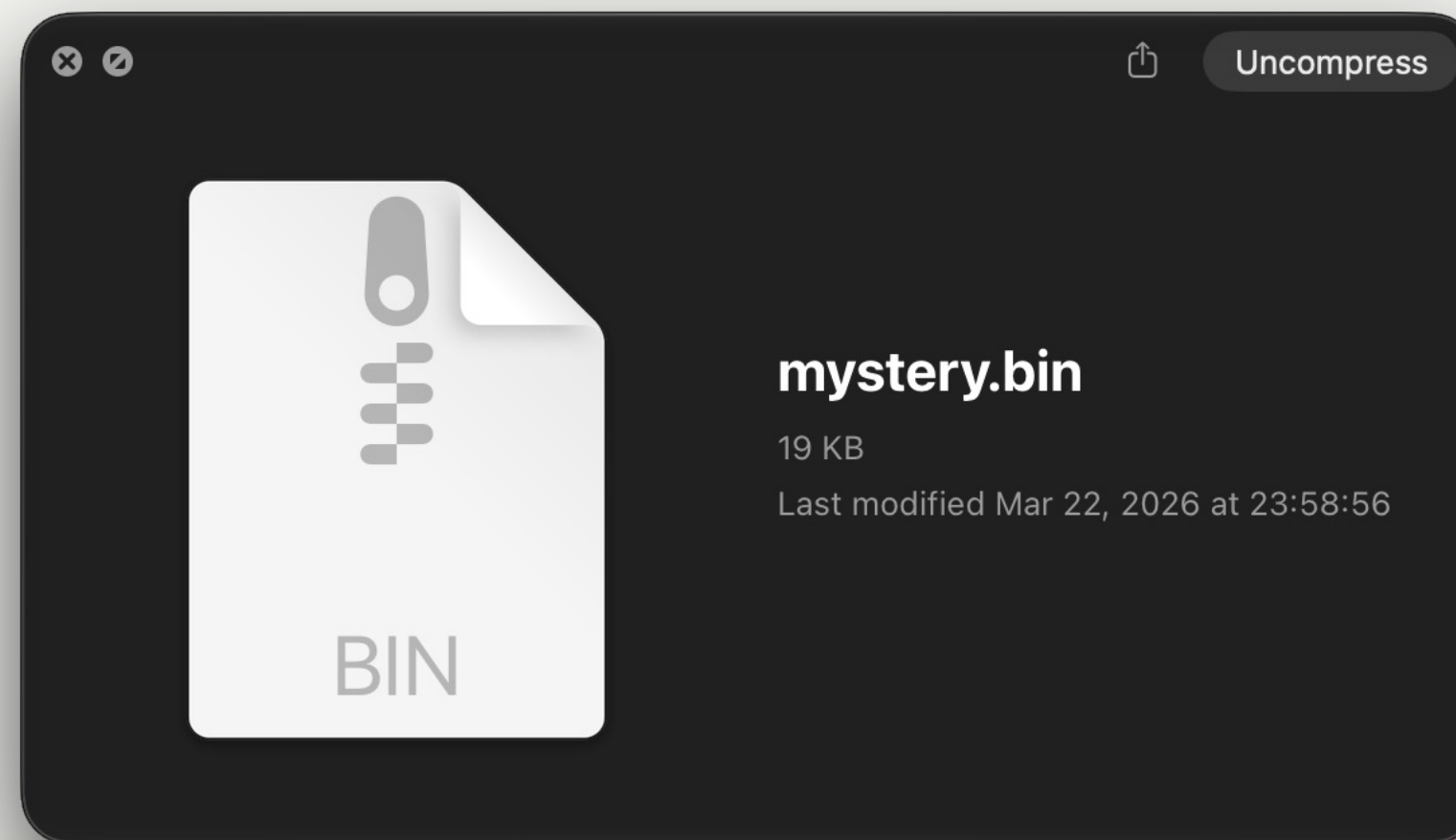


整个 64MB 的熵分布在极高和极低之间震荡

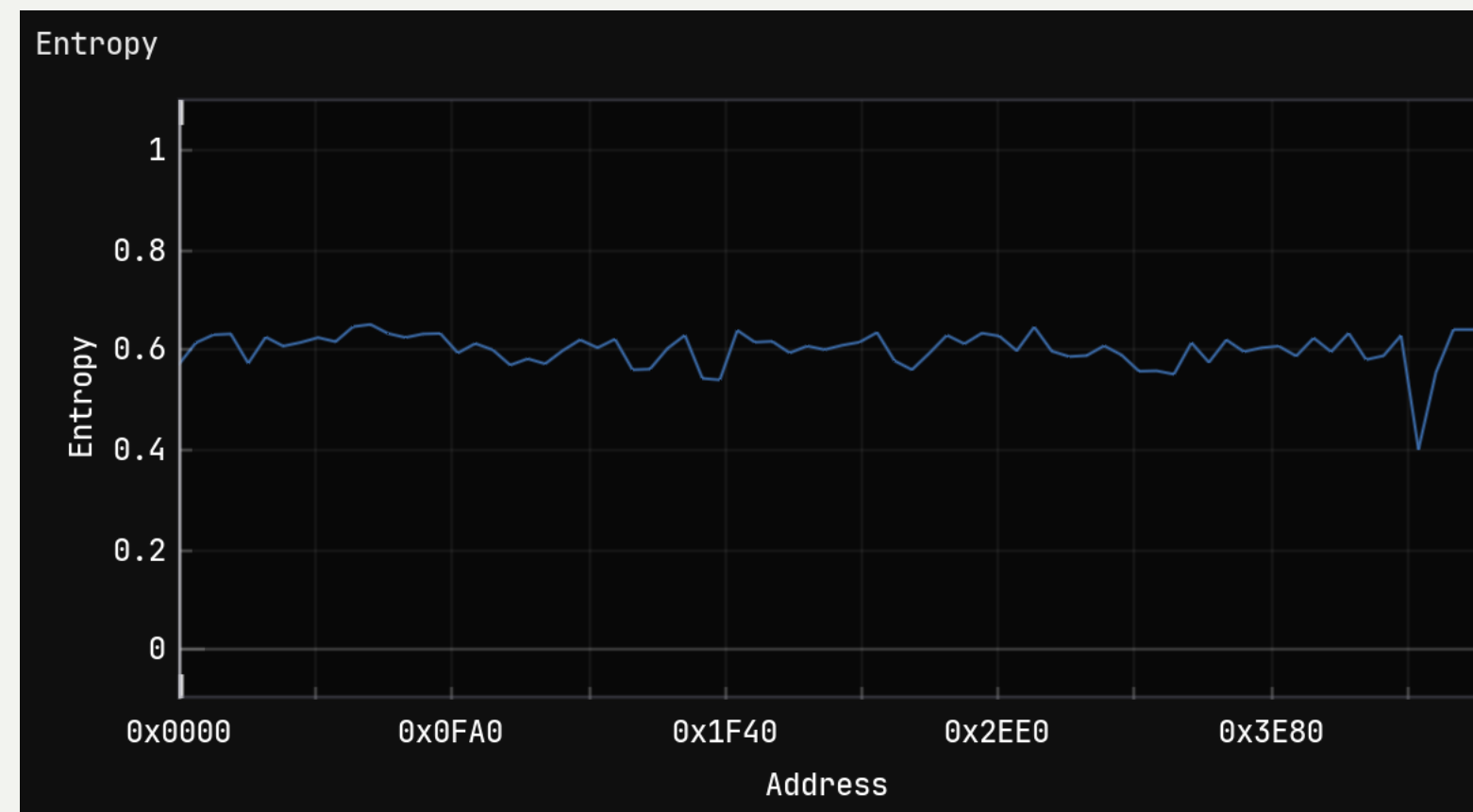


同样的一点数据重复了几千几万遍

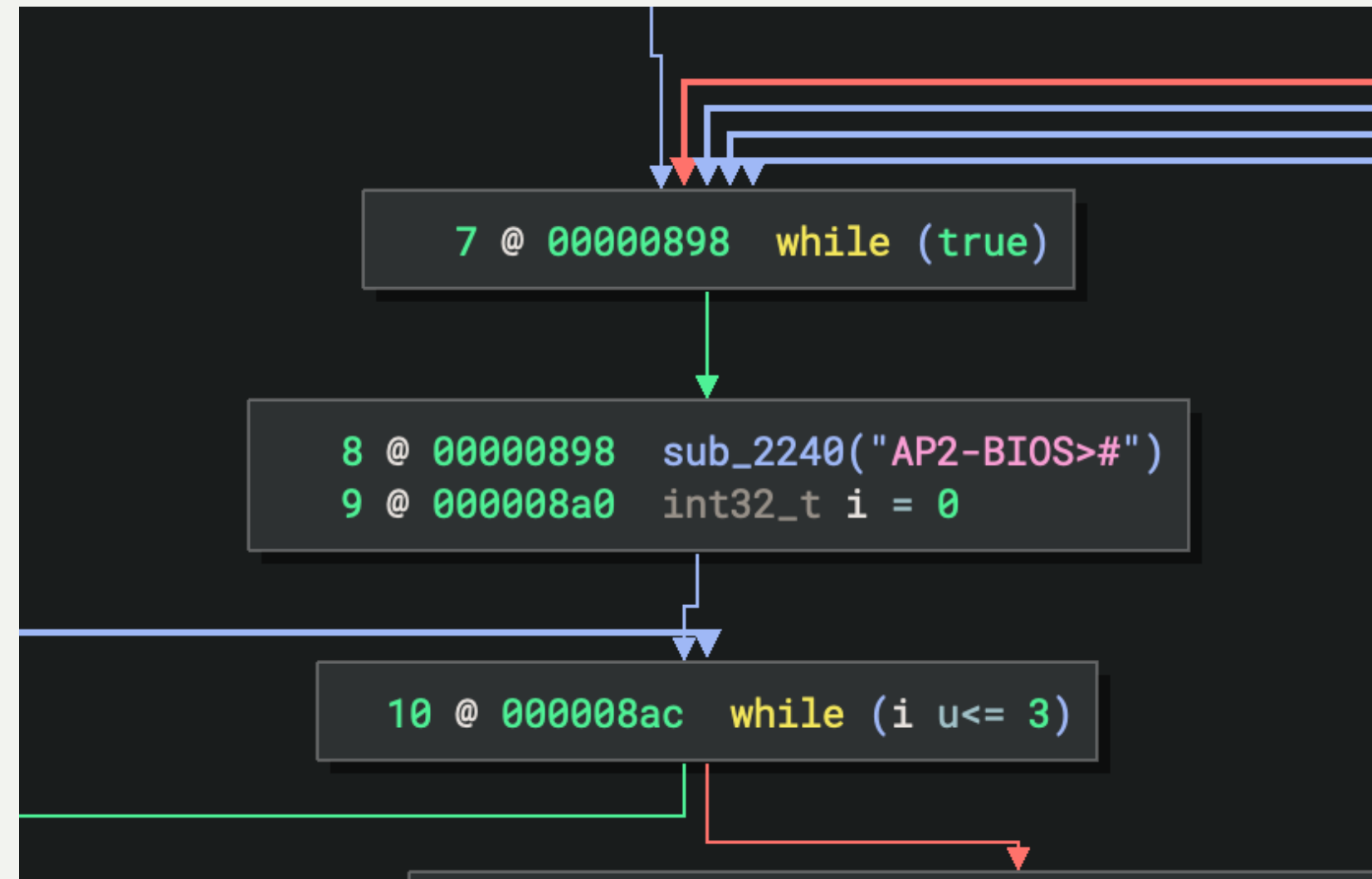
剔除重复数据后，只剩 19KB



不重复的 19KB 部分

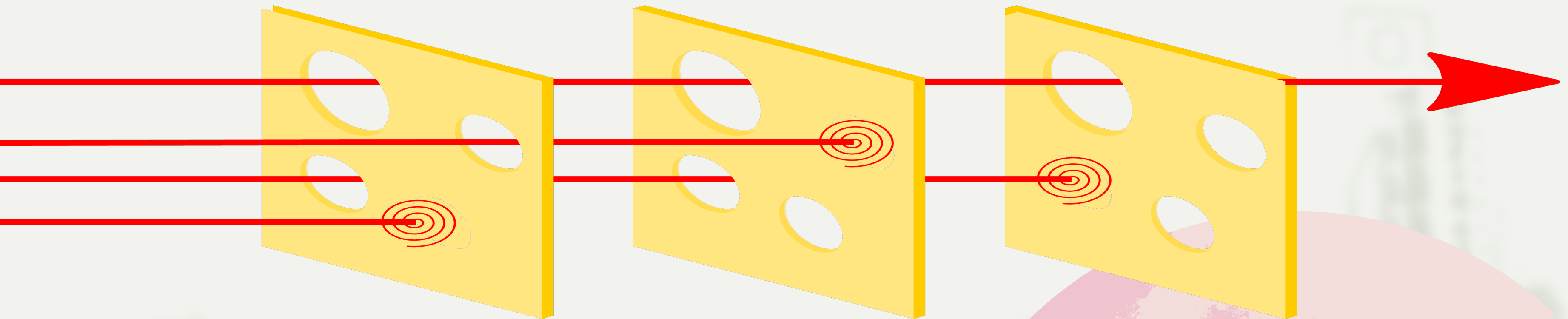


它必须、一定是可执行代码……



我好像不小心把 AIPC 的 Boot ROM 掏出来了。

发生了什么？

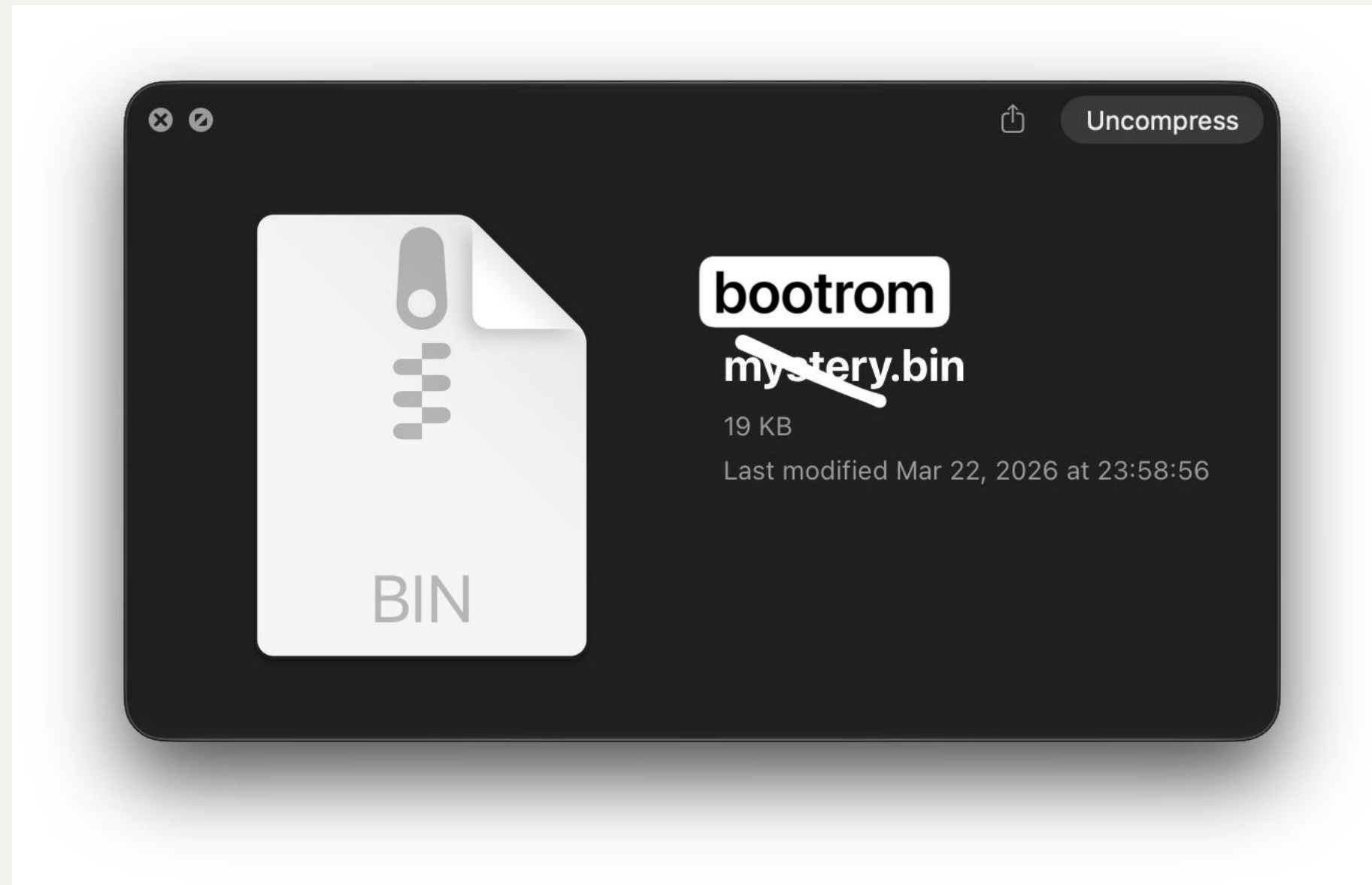


grabit 出了故障
实际在读物理地址 0x0

WinCE 5.0 不保护 0x0

AK7802 把 Boot ROM
放在 0x0

缺少任何一个巧合，就什么也得不到



第一个完全可靠的信源

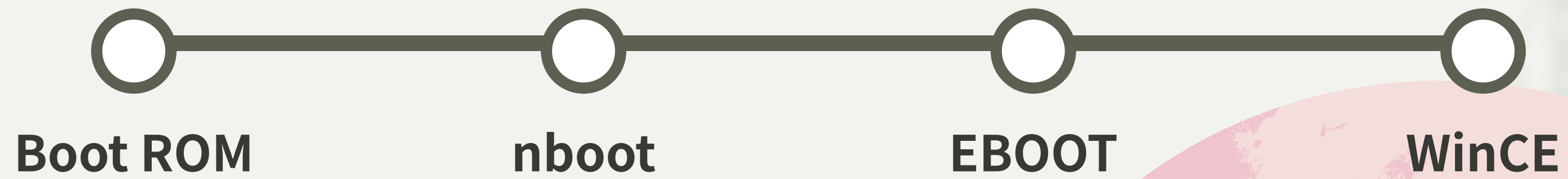
注意到:

**Boot ROM 内置了一个裸机模式
代码内部称为 USB Boot**

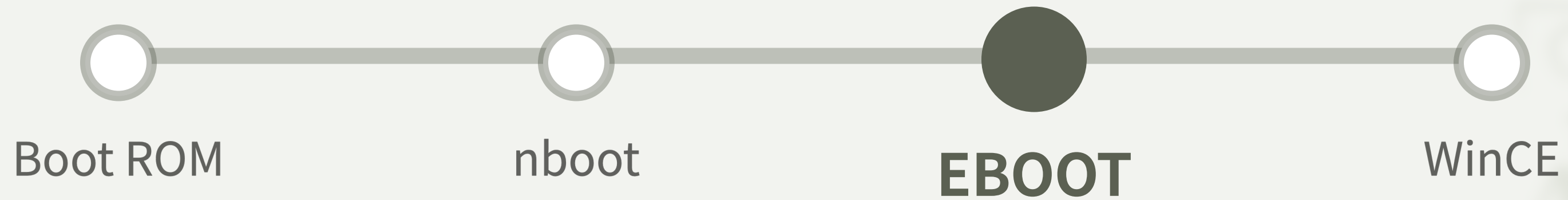
```
sub_3110:  
0 @ 00003124 int32_t var_68 = 0  
1 @ 00003128 int32_t var_64 = 0  
2 @ 0000312c int32_t var_60 = 0  
3 @ 0000313c int32_t var_88  
4 @ 0000313c __builtin_memset(dest: &var_88, ch: 0, count: 0x1c)  
5 @ 0000314c sub_1d74()  
6 @ 00003150 sub_318c()  
7 @ 00003158 sub_2240("\nAspen2_Usbboot>#")
```

从 PC 通过 USB 可以任意读写内存，跳转执行任意地址

目标已更新：沿着引导链往下走



首先，要拿到 EBOOT!



EBOOT: WinCE 的引导器（包含键盘驱动）

代码量比整个 WinCE 固件少得多

用 USB Boot 的读写能力制作出 NAND 提取工具

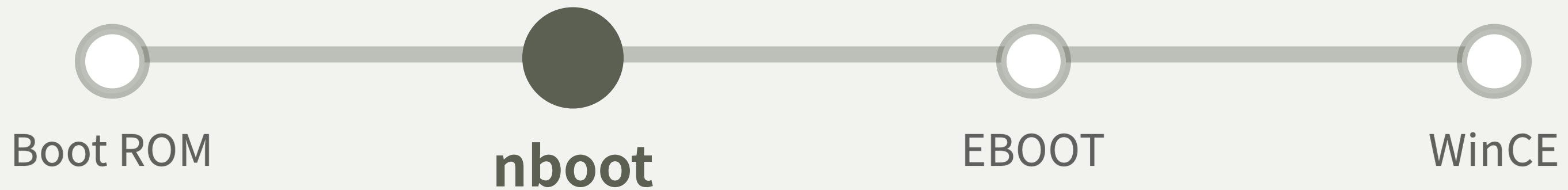
aipc-nand-dump

Full NAND dump tool for AIPC.

Dumps the entire 512 MB NAND flash including OOB/ECC data in about 15 minutes via USB boot mode. The dump can be used to restore the factory system or as a backup before flashing custom firmware. Keep it safe.

This tool is AIPC-specific (requires DDR init and uses AIPC's NAND header for timing). For a chip-generic AK7802 NAND dump tool, see [tools/nand-dump-min](#).

把整块闪存 dump 下来



引导链的第二级 nboot，只有 4KB，反编译轻松愉快

Name	Address
nboot_relocate_and_enter	0x00000000
hang_forever	0x00000060
nboot_main	0x000000cc
uart_init	0x00000124
uart_putc	0x00000170
l2_bind_nf_dma	0x000001ec
l2_copy_from_buf0	0x0000022c
nf_classify_dma_result	0x000002ec
ecc_fix_chunk_from_status	0x00000330
ecc_fix_page_from_hw_regs	0x00000480
nf_seq_start	0x000004c8
nf_seq_wait_done	0x000004e8
nf_emit_addr_cycles	0x00000504
nf_read_page_with_ecc	0x00000598
nf_read_1chunk_page	0x00000754
nf_read_4chunk_page	0x00000774
nf_read_8chunk_page	0x00000794
nf_read_raw_range	0x000007b4
nboot_init_nand_params	0x000008dc
nboot_read_page_or_meta	0x00000994
nboot_classify_block	0x00000b50
nboot_load_eboot	0x00000c04


```
000000cc void nboot_main() __noreturn
000000d8     uart_init()
000000e0     uart_putc(0x53)
000000e4     nboot_init_nand_params()
000000ec     uart_putc(0x4c)
00000104     nboot_load_eboot(0x30037fd4, 2, 0x64000)
0000010c     uart_putc(0x42)
0000011c     0x30038000() // jump to EBOOT
0000011c
00000120     while (true)
00000120     |     nop

00000124 int32_t uart_init()
00000130     *0x8000078 |= 0x200
00000150     *0x2002c040 |= 0x800
00000164     *0x20026000 = 0x30200208
00000168     *0x2002600c = 0
0000016c     return 0x20026000
```

nboot 就做这么点事，没有我们关心的东西



胜利在望，从 NAND dump 中抠出 EBOOT
好像变得像呼吸一样简单了

但是 EBOOT 的反编译结果难以理解

```
00028414      int32_t sub_28414(uint8_t* arg1, int16_t* arg2, int16_t arg3, int64_t
00028414      void* arg8 @ r11)

00028414 00000000 andeq    r0, r0, r0
00028418 c0bacd78 stmvc    sp, {r6, r7, r9, r11, r12, sp, pc} ^ {arg_0} {arg_4}
0002841c 188a4bd0 suble    r8, r11, r8, lsl r10
00028420 d0b10380 ldrdhi   r11, r12, [r3], -r0
00028424 08b00380 andhi    r11, r3, r8
00028428 70b10380 andhi    r11, r3, r0, ror r1
0002842c a06b1080 and.shi  r6, r0, r0, lsr #0x17
00028430 9cb10380 mulhi    r3, r12, r1
00028434 58000f80 andhi    r0, pc, r8, asr r0
00028438 01010100 andeq    r0, r1, r1, lsl #0x2
0002843c c4b00380 andhi    r11, r3, r4, asr #0x1
00028440 64590f80 andhi    r5, pc, r4, ror #0x12
00028444 04b20380 andhi    r11, r3, r4, lsl #0x4
00028448 38b20380 andhi    r11, r3, r8, lsr r2
```

甚至连函数入口都不规律

先后怀疑过
非常规调用约定
混淆
自修改代码

.....

没有任何一种能解释这种程度的混乱

又一次陷入没有反馈的死局

不知道问题在哪
不知道该从哪里下手



但这次不一样……

现在手里有 USB Boot，可以加载任意代码

需要一个裸机程序，绕开 WinCE，
直接验证我对硬件的理解……

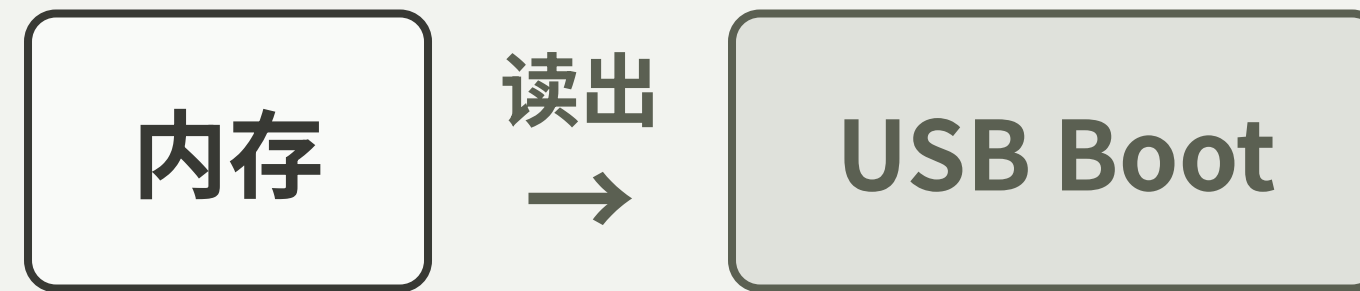
……这个程序要有画面、有交互。



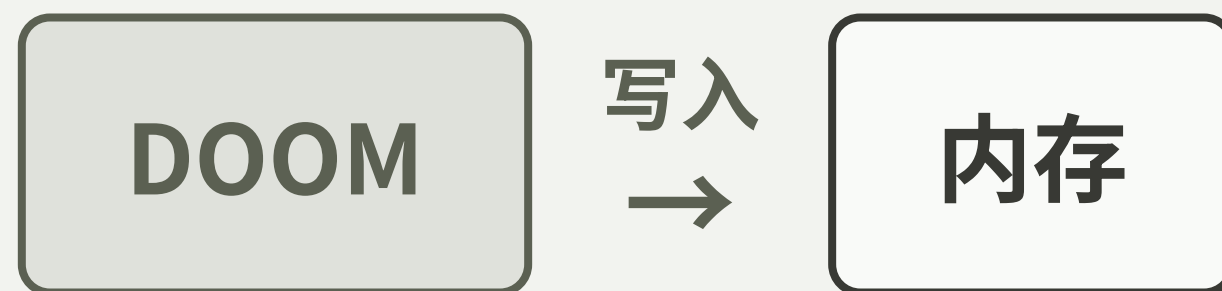
DOOM 也需要调试，调试需要日志

没有 UART，没有 JTAG，
怎么从这台机器上取回数据？





**USB Boot 可以读出内存
——但只能在上电时进入**



DOOM 可以写入内存
——但数据在断电后消失



只要断电和重新上电的时间间隔足够短
内存中的数据就可以留住

冷启动攻击



1. 把 DOOM 的日志函数实现成写入内存中的特定地址
2. DOOM 卡住时拔电源
3. 立刻插回去，进 USB Boot
4. 把那块内存 dump 出来


```
17 =====
18 | | | | | | | DOOM Shareware
19 =====
20 | Doom Generic is free software, covered by the GNU General Public
21 | License. There is NO warranty; not even for MERCHANTABILITY or FITNESS
22 | FOR A PARTICULAR PURPOSE. You are welcome to change and distribute
23 | copies under certain conditions. See the source for more information.
24 =====
25 I_Init: Setting up machine state.
26 M_Init: Init miscellaneous info.
27 R_Init: Init DOOM refresh daemon - .....
28 P_Init: Init Playloop state.
29 S_Init: Setting up sound.
30 D_CheckNetGame: Checking network game status.
31 startskill 2 deathmatch: 0 startmap: 1 startepisode: 1
32 player 1 of 1 (1 nodes)
33 Emulating the behavior of the 'Doom 1.9' executable.
```

拿到 DOOM 日志了！

这个手法成了 DOOM 支线最主要的调试手段

注意到：

**同样的方法
可以用在 Linux 上**

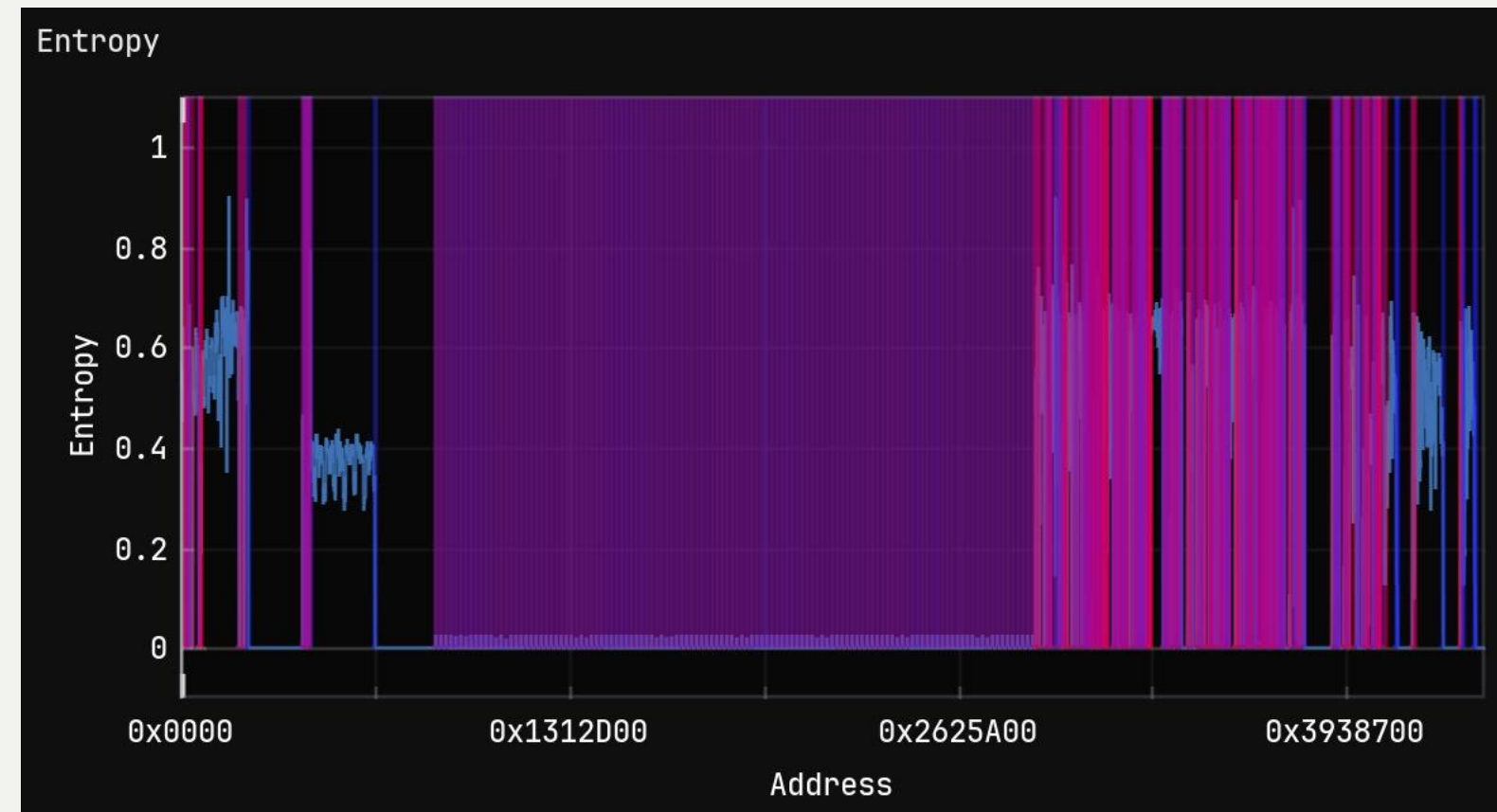
1 把 DOOM 的日志函数实现成写入内存中的特定地址

正常开机，进 WinCE

2. DOOM 卡住时 拔电源

3. 立刻插回去，进 USB Boot

4. 把 那块 全部 内存 dump 出来



注意到：

内存中还留着 EBOOT，没有被 WinCE 覆盖掉

没指望有什么不同……

这一次，反编译器给出了一份极其干净的结果

```
00029730      int32_t sub_29730(void* arg1)

00029730  30402de9  push    {r4, r5, lr} {__saved_lr} {__saved_r5} {__saved_r4}
00029734  0040a0e1  mov     r4, r0
00029738  28509fe5  ldr     r5, data_29768 {0x800ff7b0}
0002973c  543495e5  ldr     r3, [r5, #0x454] {0x800ffc04}
00029740  000053e3  cmp     r3, #0
00029744  62f9ff1b  blne    sub_27cd4
00029748  0030a0e3  mov     r3, #0
0002974c  5c3485e5  str     r3, [r5, #0x45c] {sub_0} {0x800ffc0c}
00029750  544485e5  str     r4, [r5, #0x454] {0x800ffc04}
00029754  083094e5  ldr     r3, [r4, #0x8]
00029758  583485e5  str     r3, [r5, #0x458] {0x800ffc08}
```


NAND dump

```
sub_28414(uint8_t* arg1, int16_t* arg2, int16_t* arg8 @ r11)
```

```
andeq    r0, r0, r0
stmvc    sp, {r6, r7, r9, r11, r12, sp, pc} ^
suble    r8, r11, r8, lsl r10
ldrdhi   r11, r12, [r3], -r0
andhi    r11, r3, r8
andhi    r11, r3, r0, ror r1
and.shi  r6, r0, r0, lsr #0x17
mulhi    r3, r12, r1
andhi    r0, pc, r8, asr r0
andeq    r0, r1, r1, lsl #0x2
```

内存 dump

```
sub_29730(void* arg1)
```

```
push     {r4, r5, lr} {__saved_lr} {__saved_r5}
mov      r4, r0
ldr      r5, data_29768 {0x800ff7b0}
ldr      r3, [r5, #0x454] {0x800ffc04}
cmp      r3, #0
blne     sub_27cd4
mov      r3, #0
str      r3, [r5, #0x45c] {sub_0} {0x800ffc04}
str      r4, [r5, #0x454] {0x800ffc04}
ldr      r3, [r4, #0x8]
str      r3, [r5, #0x458] {0x800ffc08}
```

NAND dump 中的那份 EBOOT，是损坏的。

一般的 NAND，每页 2112 字节：2048 数据 + 64 校验区

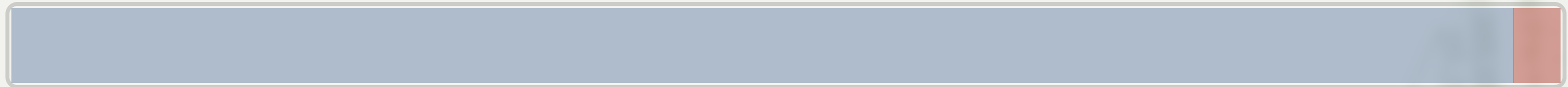


2048 bytes 数据

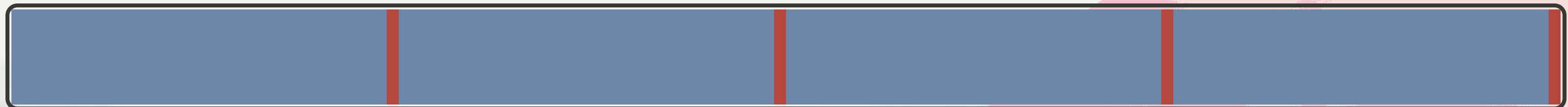
读数据时，只需取前 2048 字节

但 AIPC 不是这样

脑回路正常的 NAND 页布局

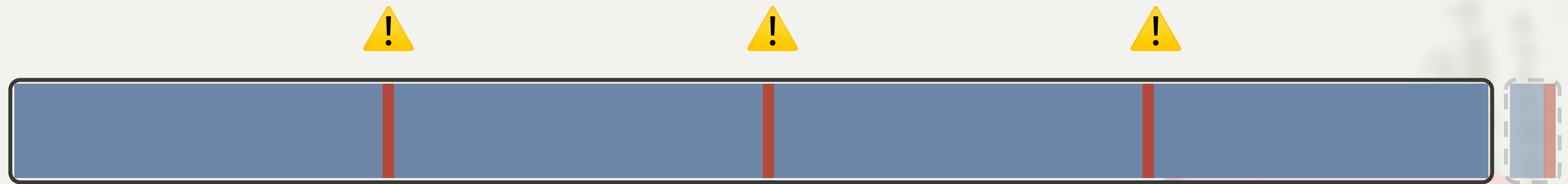


AIPC 的实际布局



每 512 字节数据后紧跟 16 字节 ECC，一页重复四次

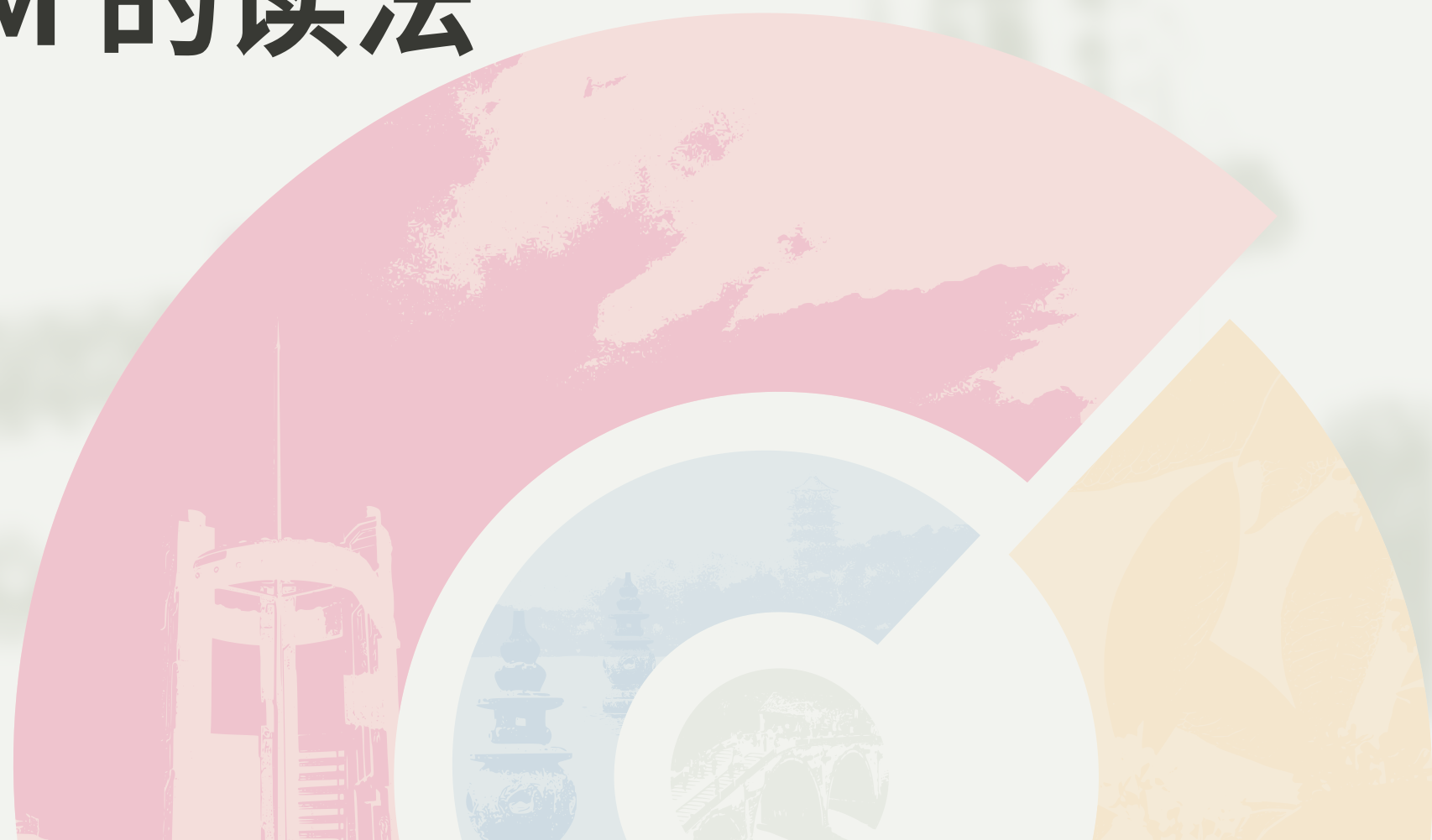
如果用 Boot ROM 的读法，只会取前 2048 字节



末尾 64 字节丢失 ↗

三段 ECC 校验字节会直接混进数据

**而我的提取工具
就是 Boot ROM 的读法**

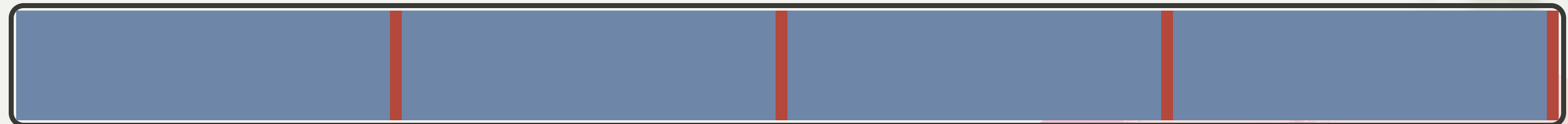


还记得 nboot 吗？

nboot:



EBOOT:



nboot 不使用 ECC
Boot ROM 的读法对 nboot 完全正确



`nf_read_page_with_ecc`



<code>hang_forever</code>	<code>0x00000000</code>
<code>nboot_main</code>	<code>0x000000cc</code>
<code>uart_init</code>	<code>0x00000124</code>
<code>uart_putc</code>	<code>0x00000170</code>
<code>l2_bind_nf_dma</code>	<code>0x000001ec</code>
<code>l2_copy_from_buf0</code>	<code>0x0000022c</code>
<code>nf_classify_dma_result</code>	<code>0x000002ec</code>
<code>ecc_fix_chunk_from_status</code>	<code>0x00000330</code>
<code>ecc_fix_page_from_hw_regs</code>	<code>0x00000480</code>
<code>nf_seq_start</code>	<code>0x000004c8</code>
<code>nf_seq_wait_done</code>	<code>0x000004e8</code>
<code>nf_emit_addr_cycles</code>	<code>0x00000504</code>
<code>nf_read_page_with_ecc</code>	<code>0x00000598</code>
<code>nf_read_1chunk_page</code>	<code>0x00000754</code>
<code>nf_read_4chunk_page</code>	<code>0x00000774</code>
<code>nf_read_8chunk_page</code>	<code>0x00000794</code>
<code>nf_read_raw_range</code>	<code>0x000007b4</code>
<code>nboot_init_nand_params</code>	<code>0x000008dc</code>
<code>nboot_read_page_or_meta</code>	<code>0x00000994</code>
<code>nboot_classify_block</code>	<code>0x00000b50</code>
<code>nboot_load_eboot</code>	<code>0x00000c04</code>

但 nboot 做的事情之一，就是用带 ECC 纠错的方式加载 EBOOT

答案一直在 nboot 里

终于拿到了日思夜想的 键盘驱动（还有 LCD 初始化序列）

SPI Controller Used By CH374

`ch374_init` calls `spi_init_controller(0)`, which selects physical `0x20024000` and stores its virtual base in `0x80107768`. The numbering that the surrounding documentation uses is still unresolved. This page records only the physical base that EBOOT selects.

The transfer helpers use these offsets, relative to the selected SPI base:

Offset	Use in EBOOT
+0x00	control
+0x04	status
+0x0C	transfer count
+0x10	cleared before TX
+0x14	cleared before RX
+0x18	TX data
+0x1C	RX data
+0x20	written once by <code>spi_config_mode_clock</code> to <code>0x00FFFFFF</code>

移植进 DOOM，终于有了画面和输入



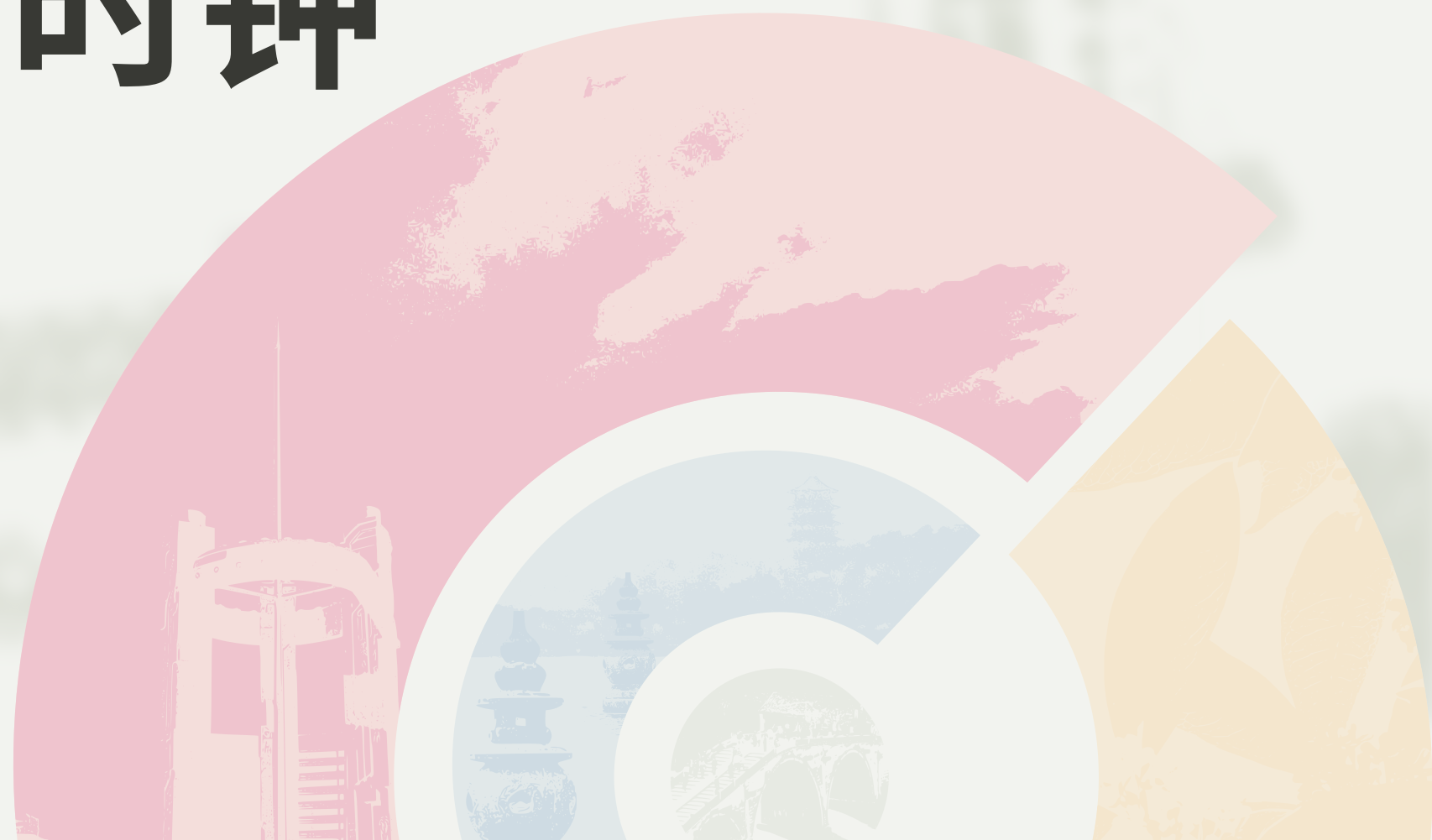
• • • • •



画面卡在第一屏



又是……时钟



但这次不一样……

有了 USB Boot，可以在裸机环境中像做物理实验一样

提出假设



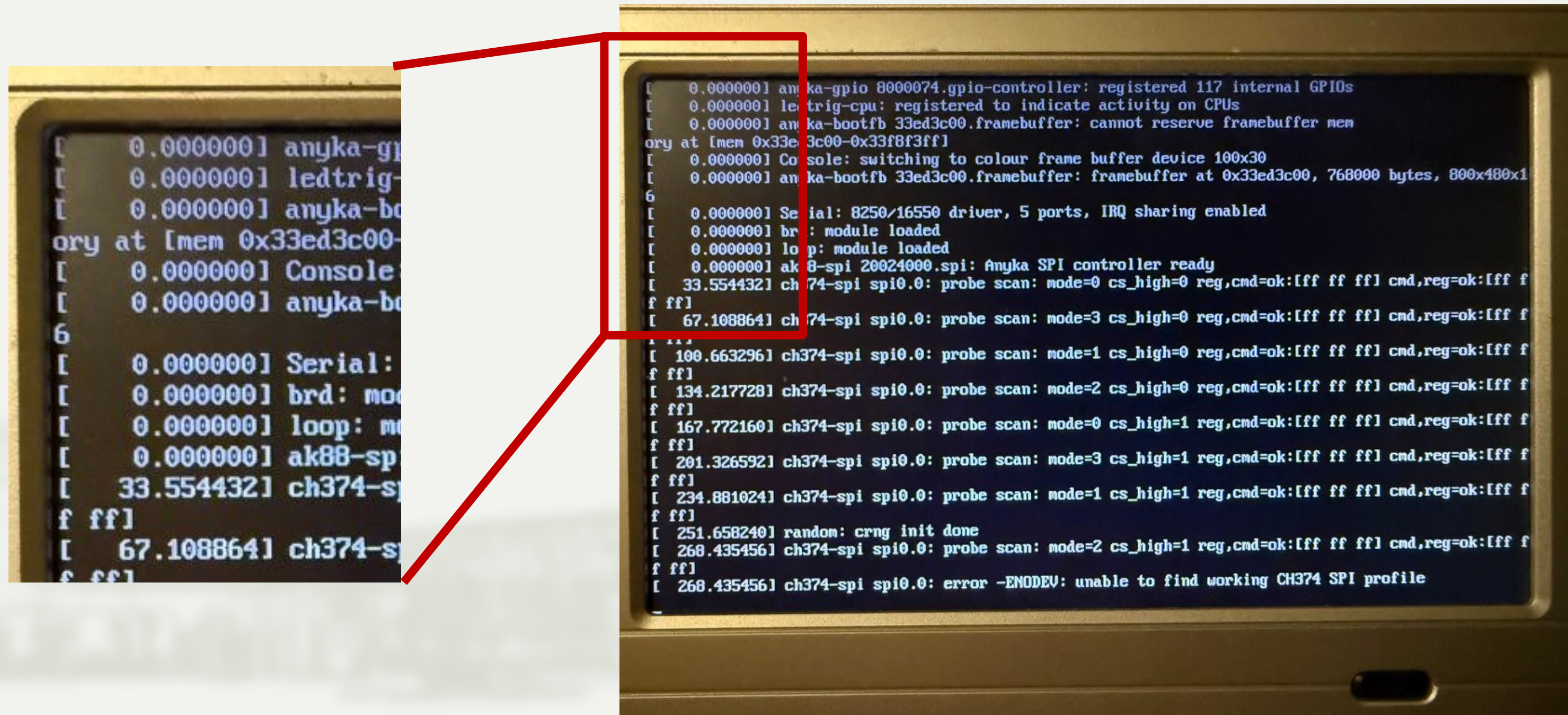
设计实验



验证猜想

……由 AI 自动迭代！

稍早前.....



```
[ 0.000000] anyka-gpio 8000074.gpio-controller: registered 117 internal GPIOs
[ 0.000000] ledtrig-cpu: registered to indicate activity on CPUs
[ 0.000000] anyka-bootfb 33ed3c00.framebuffer: cannot reserve framebuffer mem
ory at [mem 0x33ed3c00-0x33f8f3ff]
[ 0.000000] Console: switching to colour frame buffer device 100x30
[ 0.000000] anyka-bootfb 33ed3c00.framebuffer: framebuffer at 0x33ed3c00, 768000 bytes, 800x480x1
6
[ 0.000000] Serial: 8250/16550 driver, 5 ports, IRQ sharing enabled
[ 0.000000] brd: module loaded
[ 0.000000] loop: module loaded
[ 0.000000] ak88-spi 20024000.spi: Anyka SPI controller ready
[ 33.554432] ch374-spi spi0.0: probe scan: mode=0 cs_high=0 reg,cmd=ok:[ff ff ff] cmd,reg=ok:[ff f
f ff]
[ 67.108864] ch374-spi spi0.0: probe scan: mode=3 cs_high=0 reg,cmd=ok:[ff ff ff] cmd,reg=ok:[ff f
f ff]
[ 100.663296] ch374-spi spi0.0: probe scan: mode=1 cs_high=0 reg,cmd=ok:[ff ff ff] cmd,reg=ok:[ff f
f ff]
[ 134.217728] ch374-spi spi0.0: probe scan: mode=2 cs_high=0 reg,cmd=ok:[ff ff ff] cmd,reg=ok:[ff f
f ff]
[ 167.772160] ch374-spi spi0.0: probe scan: mode=0 cs_high=1 reg,cmd=ok:[ff ff ff] cmd,reg=ok:[ff f
f ff]
[ 201.326592] ch374-spi spi0.0: probe scan: mode=3 cs_high=1 reg,cmd=ok:[ff ff ff] cmd,reg=ok:[ff f
f ff]
[ 234.881024] ch374-spi spi0.0: probe scan: mode=1 cs_high=1 reg,cmd=ok:[ff ff ff] cmd,reg=ok:[ff f
f ff]
[ 251.658240] random: crng init done
[ 268.435456] ch374-spi spi0.0: probe scan: mode=2 cs_high=1 reg,cmd=ok:[ff ff ff] cmd,reg=ok:[ff f
f ff]
[ 268.435456] ch374-spi spi0.0: error -ENODEV: unable to find working CH374 SPI profile
```


[0.000000]
[0.000000]
[0.000000]
[0.000000]
[0.000000]
[33.554432]

时间是……离散的？

一个定时器，两个寄存器

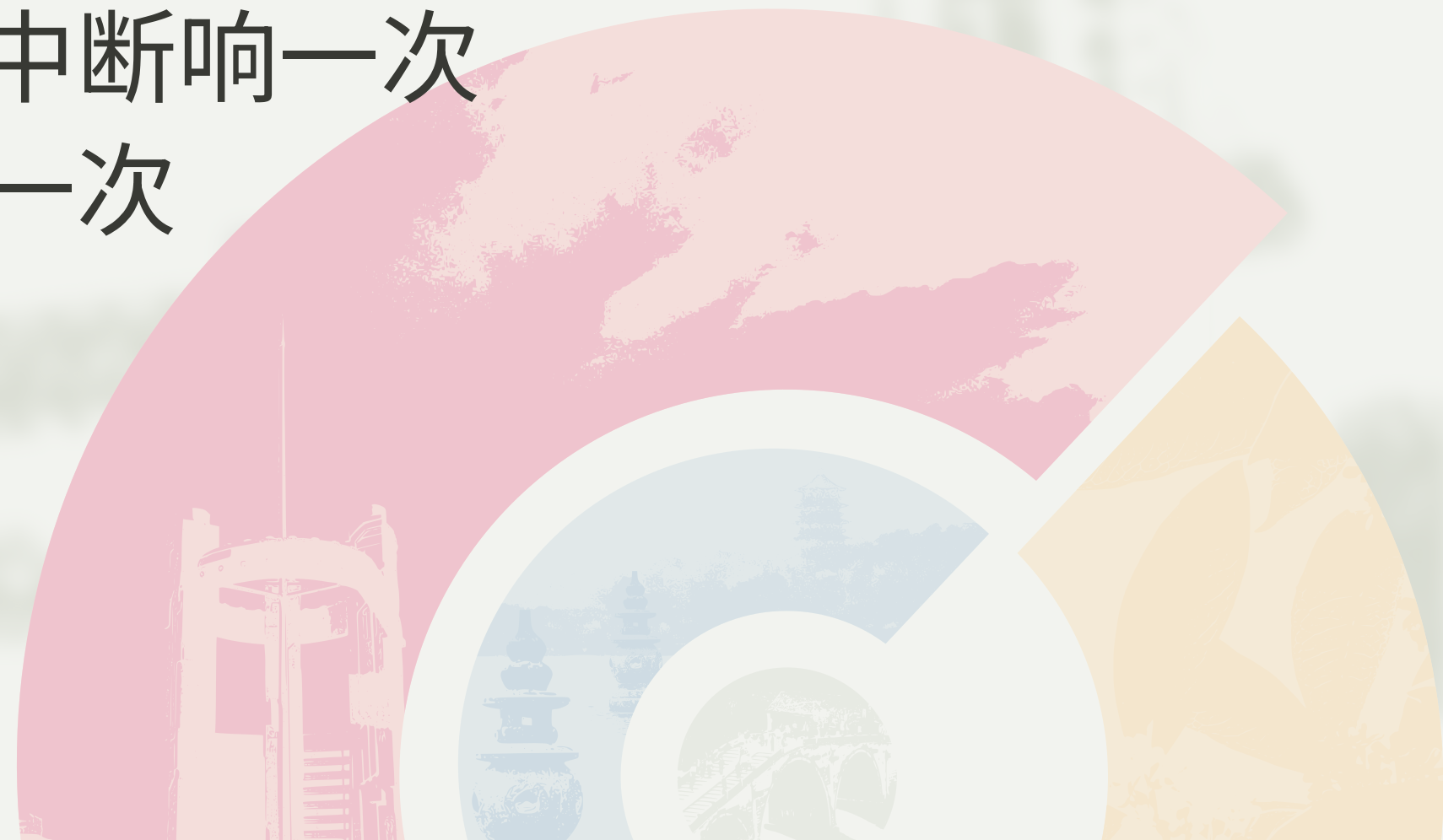
`TIMER2_RDBACK` 正在前进的时间

`TIMER2_CTRL` 我们写进去的控制参数

驱动读的是 `TIMER2_CTRL`

于是内核只能靠溢出中断感知时间

计时器跑完一圈，中断响一次
时间才前进一次

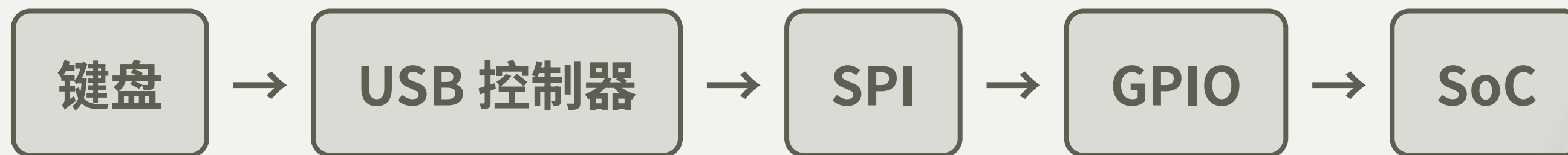


「往后，再不会有什么变数了」

**一个月前，键盘不工作
SPI 总线像断了一样**

内核的时间每隔几秒才会前进一次

所有的总线通信在这种时序下都不可能正常工作



它们只是一直在等一个正确的时钟

时钟修好了，DOOM 可以玩了



回到主线剧情

```
[ 4.517079] hub control -> typeReq: 0000 wValue: 0000 wIndex: 0000
[ 4.522957] hub control -> wPortStatus[0]: 0203 wPortChange: 10
[ 4.529504] hub control -> typeReq: 2301 wValue: 0014 wIndex: 0001
[ 4.535643] hub control -> wPortStatus[0]: 0203 wPortChange: 10
[ 5.738485] input: HID 04d9:1203 as /devices/platform/20024000.spi/spi_master/spi0/spi0.0/usb1/1-1/1-1:1.0/0003:04D9:1203.0001/input/input0
[ 5.815681] hid-generic 0003:04D9:1203.0001: input: USB HID v1.11 Keyboard [HID 04d9:1203] on usb-
spi0.0-1/input0
[ 6.381965] input: HID 04d9:1203 System Control as /devices/platform/20024000.spi/spi_master/spi0
/spi0.0/usb1/1-1/1-1:1.1/0003:04D9:1203.0002/input/input1
[ 6.458670] input: HID 04d9:1203 Consumer Control as /devices/platform/20024000.spi/spi_master/spi0
/spi0.0/usb1/1-1/1-1:1.1/0003:04D9:1203.0002/input/input2
[ 6.476113] hid-generic 0003:04D9:1203.0002: input: USB HID v1.11 Device [HID 04d9:1203] on usb-s
pi0.0-1/input1
h
cttyhack: can't open '/dev/tty0': No such file or directory
/bin/sh: can't access tty: job control turned off
~ # id
uid=0 gid=0
~ # who
~ # echo hi
hi
~ # _
```

时钟修好后，Linux 的键盘也终于工作了

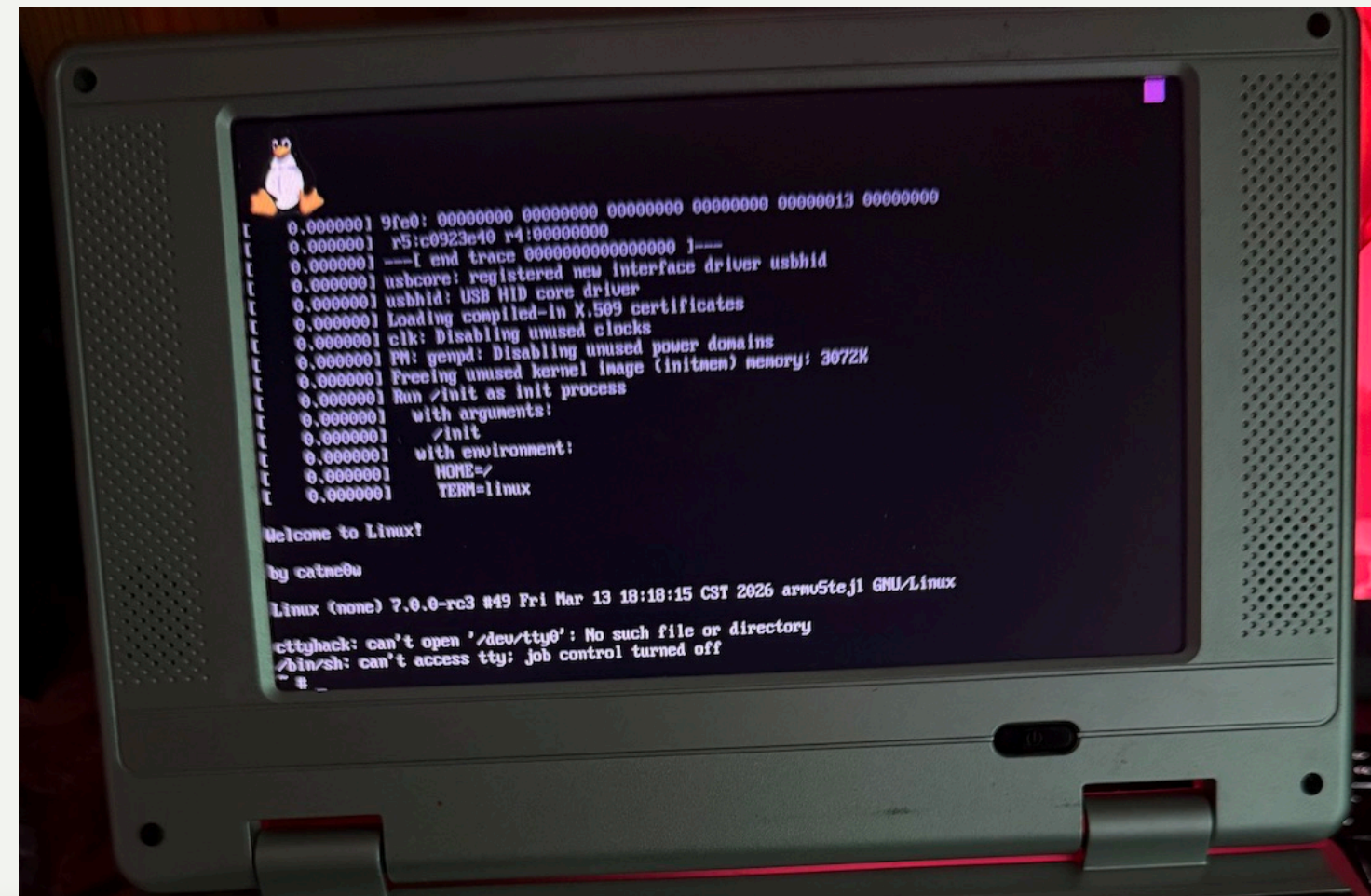
DOOM 支线



Linux 内核

DOOM 支线攒下的所有成果，全部灌回 Linux 内核

2026-03



Linux 就在眼前，但没有任何办法交互

2026-08



AIPC 成功点亮星霞 OS

十几年前的那个心愿，
终于实现了。

fin



致谢

Cyano - 星霞 OS

Gray - 测试

特别鸣谢

AOSC 社区

FurLibrary 论坛交流群 a.k.a. 动物园里的动物都是福瑞扮的

项目链接（快来点 star

github.com/CatMe0w/aipc-os

aipc-os.catme0w.org

展台区有 AIPC 实机，快去看看吧